Desarrollo de API REST con FastAPI:

CRUD, validación de datos y organización modular

Evidencias GA2-AA1-EV01, GA2-AA1-EV02, GA2-AA2-EV03 y GA2-AA3-EV04

Integrantes

Juan Esteban Lezcano Tejada

Mariana Suárez

Emanuel Villa

Servicio Nacional de Aprendizaje (SENA)

Análisis y Desarrollo de Software (ADSO)

Medellín

Instructor: Mateo Arroyave

8 de noviembre de 2026

Índice

Introducción	3
Objetivos	4
Desarrollo de la actividad	5
GA2-AA1-EV01. Cuestionario de reflexión inicial	5
GA2-AA1-EV02. Cuadro conceptual y mapa conceptual	6
GA2-AA2-EV03. Laboratorio CRUD de productos	8
Arquitectura modular, modelos y endpoints	8
Pruebas y evidencias del recurso productos	9
GA2-AA3-EV04. CRUD de categorías	18
Pruebas y evidencias del recurso categorías	19
Análisis de resultados y comprobación técnica	28
Código fuente de la API modular	29
Conclusiones	31
Referencias bibliográficas	32
Control de versiones	33

Introducción

Una API REST permite que distintas aplicaciones consulten y administren recursos por medio del protocolo HTTP. En la Guía 1 se implementaron endpoints de lectura y se exploró la documentación interactiva de FastAPI. La Guía 2 retoma el catálogo de una tienda y amplía su funcionalidad para permitir la creación, consulta, actualización y eliminación de productos y categorías.

La actividad utiliza modelos de Pydantic para describir y validar los cuerpos de las solicitudes, y APIRouter para distribuir los endpoints en módulos independientes. Esta estructura facilita la comprensión del código y el trabajo colaborativo, manteniendo los datos de ejemplo en listas de Python; no se implementa todavía una base de datos persistente (Ramírez, 2024a, 2024b).

El informe presenta las cuatro evidencias definidas por la Guía 2: reflexión inicial, cuadro y mapa conceptual, laboratorio CRUD de productos y reto de categorías. Para las evidencias prácticas se incluyen capturas reales de Swagger UI tomadas durante la ejecución local de la aplicación y la interpretación de los códigos HTTP obtenidos.

Objetivos

Objetivo general

Implementar y comprobar una API REST de tienda con FastAPI, organizada en módulos independientes y capaz de administrar productos y categorías mediante operaciones CRUD y validación automática de los datos recibidos.

Objetivos específicos

- Reconocer los problemas de mantenimiento asociados a concentrar múltiples funcionalidades en un solo archivo.
- Relacionar las operaciones crear, leer, actualizar y eliminar con los métodos HTTP y códigos de respuesta correspondientes.
- Construir modelos de entrada a partir de BaseModel y organizar los endpoints con APIRouter, prefix y tags.
- Ejecutar y documentar las operaciones CRUD de productos y categorías, incluidas las respuestas de error 404 y 422.
- Conservar el código y la documentación en el repositorio académico único, con ejecución independiente de las guías.

Desarrollo de la actividad

GA2-AA1-EV01. Cuestionario de reflexión inicial

1. Archivo de 800 líneas. Buscar fallos sería más lento y distintos integrantes podrían modificar las mismas funciones, causando conflictos. Separar el código por recursos facilita su mantenimiento.

2. Validación del formulario. El frontend debe informar los errores al usuario; sin embargo, el servidor siempre debe comprobar los datos recibidos. FastAPI puede validarlos mediante Pydantic.

3. Operaciones necesarias en una tienda. El administrador necesita registrar productos, consultar el catálogo, modificar precios o nombres y eliminar registros cuando corresponda.

4. Relación CRUD y HTTP. Crear corresponde a POST; leer, a GET; actualizar, a PUT; y eliminar, a DELETE. Cada método expresa la acción sobre el recurso.

GA2-AA1-EV02. Cuadro conceptual y mapa conceptual

El conjunto CRUD reúne las operaciones fundamentales para administrar recursos. La Tabla 1 presenta su relación con los métodos HTTP y los códigos de éxito definidos en la guía de aprendizaje (Arroyave, 2025).

Tabla 1

Correspondencia entre operaciones CRUD, métodos HTTP y respuestas de éxito

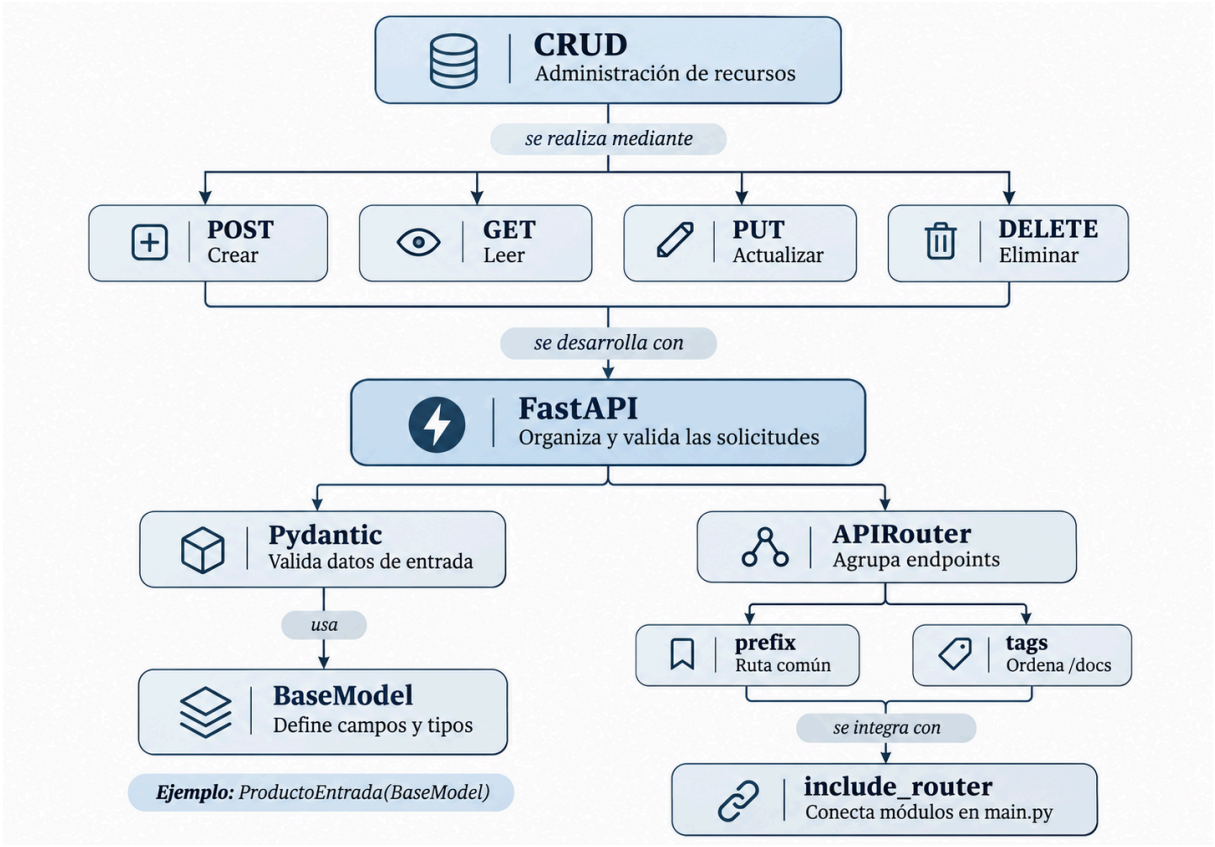
Operación	Método HTTP	Ejemplo en la API	Éxito
Crear (Create)	POST	POST /productos	201 Created
Leer (Read)	GET	GET /productos/5	200 OK
Actualizar (Update)	PUT	PUT /productos/5	200 OK
Eliminar (Delete)	DELETE	DELETE /productos/5	200 OK

Nota. Elaboración propia con base en los métodos y respuestas indicados en la Guía 2 y en la documentación HTTP de MDN Web Docs.

Los modelos Pydantic basados en BaseModel describen los campos del cuerpo de una solicitud; FastAPI valida automáticamente los datos antes de ejecutar el endpoint. APIRouter permite definir rutas por recurso, mientras que prefix establece la ruta común y tags clasifica las operaciones en Swagger UI (Ramírez, 2024a, 2024b; Pydantic, s. f.).

Figura 1

Relación entre CRUD, validación Pydantic y organización modular de la API



Nota. Elaboración propia. Los métodos HTTP materializan el CRUD, BaseModel define los campos y APIRouter organiza las rutas mediante prefix y tags.

GA2-AA2-EV03. Laboratorio CRUD de productos

El laboratorio consistió en extender el catálogo de la Guía 1 con operaciones de creación, lectura, actualización y eliminación. El módulo productos.py conserva una lista de tres productos iniciales e incorpora el modelo ProductoEntrada con los campos nombre: str, precio: float y categoria: str. El identificador lo genera el servidor y no se solicita al cliente.

Arquitectura modular, modelos y endpoints

La aplicación se distribuyó en guia_02/tienda_api/main.py y en los routers productos.py y categorias.py. El archivo principal crea FastAPI, incluye ambos routers y ofrece la ruta inicial. En cada router, prefix evita repetir /productos o /categorias y tags agrupa las operaciones en Swagger UI. La información permanece en listas de Python durante la sesión del servidor. Para ejecutar esta estructura desde la raíz del repositorio se utiliza el comando uvicorn guia_02.tienda_api.main:app --reload.

Tabla 2

Operaciones implementadas en el recurso productos

Operación	Ruta	Respuesta esperada
GET	/productos	Listado · 200
GET	/productos/{producto_id}	Producto · 200 / 404
POST	/productos	Creación · 201 / 422
PUT	/productos/{producto_id}	Actualización · 200 / 404 / 422
DELETE	/productos/{producto_id}	Eliminación · 200 / 404

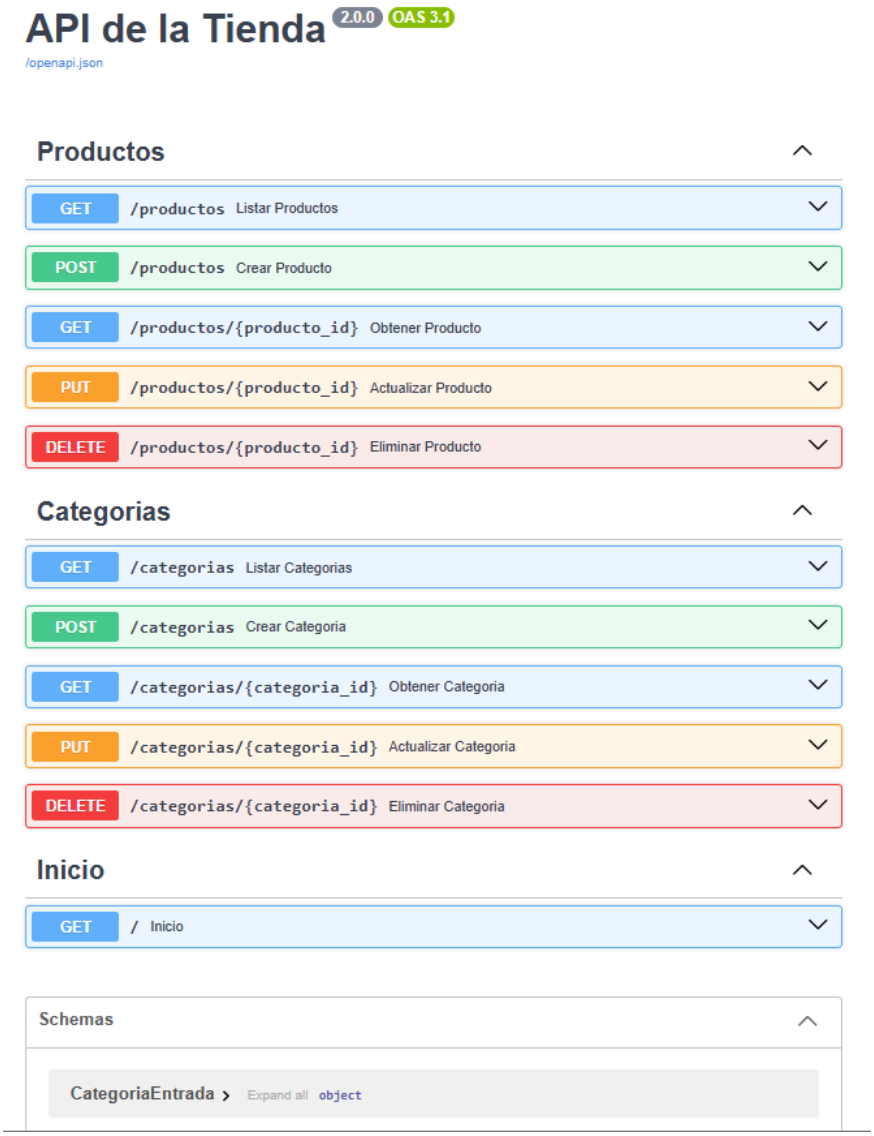
Nota. Elaboración propia a partir del router de productos publicado en el repositorio fastapi-sena.

Las pruebas se realizaron desde http://127.0.0.1:8000/docs con el servidor Uvicorn activo. Se registraron dos productos de prueba, se consultaron y posteriormente se modificaron o eliminaron, verificando las respuestas del servidor.

Pruebas y evidencias del recurso productos

Figura 2

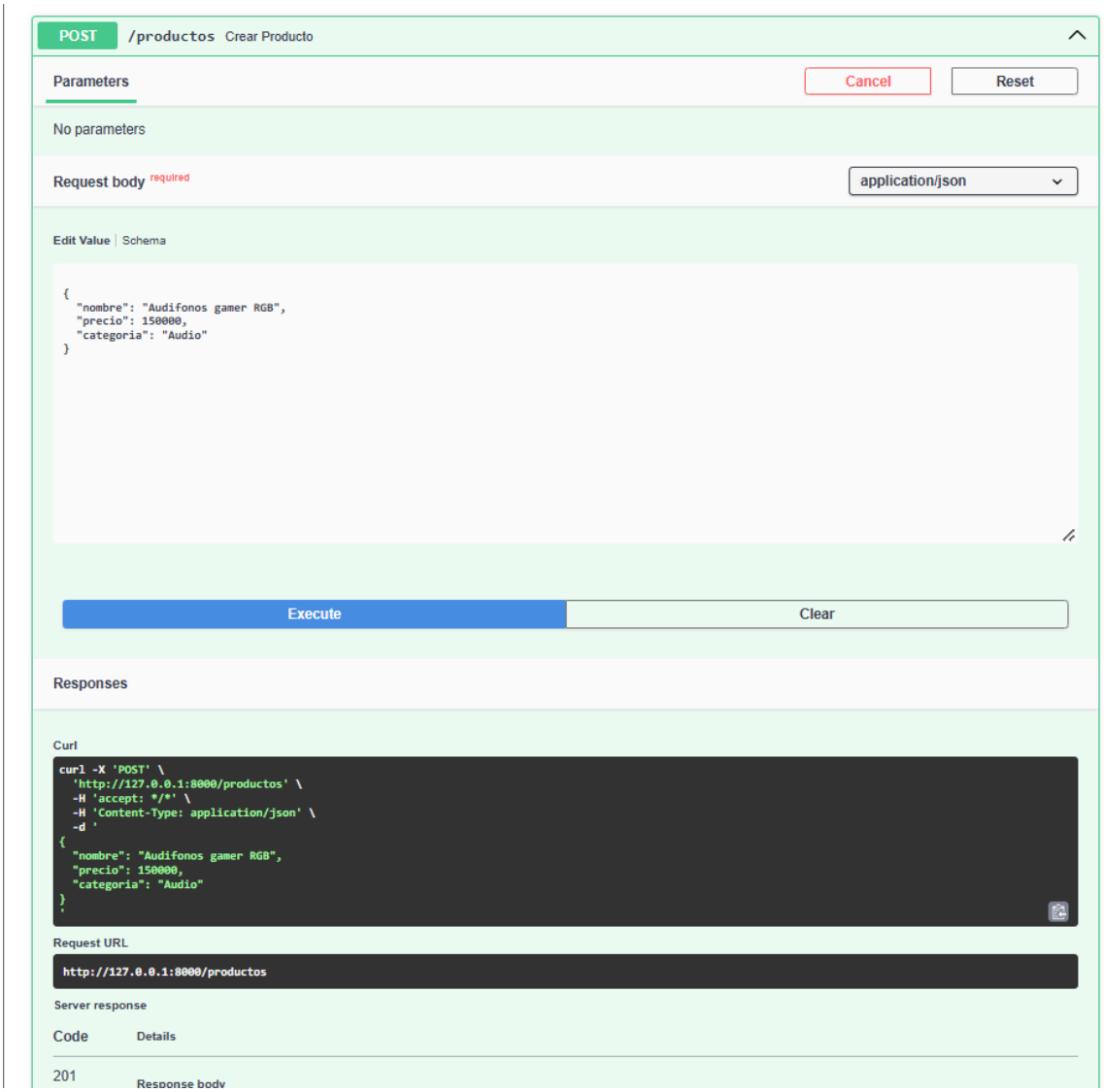
Swagger UI de la API de tienda con endpoints agrupados



Nota. Se observan las secciones Productos, Categorias e Inicio; el registro de ambas familias de rutas confirma su integración mediante APIRouter.

Figura 3

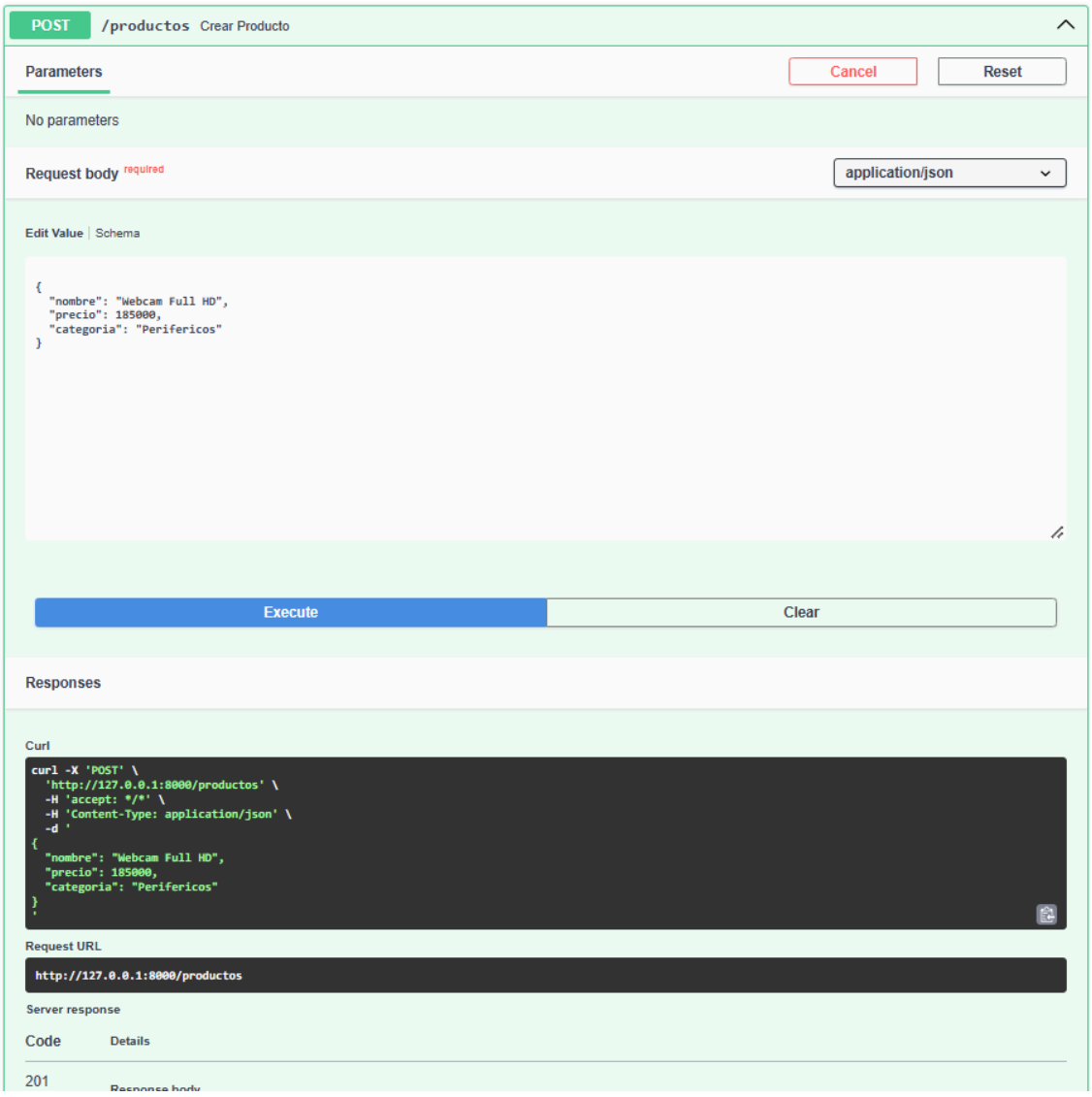
Solicitud POST para crear Audifonos gamer RGB



Nota. Se envió el cuerpo con nombre, precio 150000 y categoría Audio. Swagger UI presentó HTTP 201; el cuerpo completo de respuesta no quedó visible en este recorte.

Figura 4

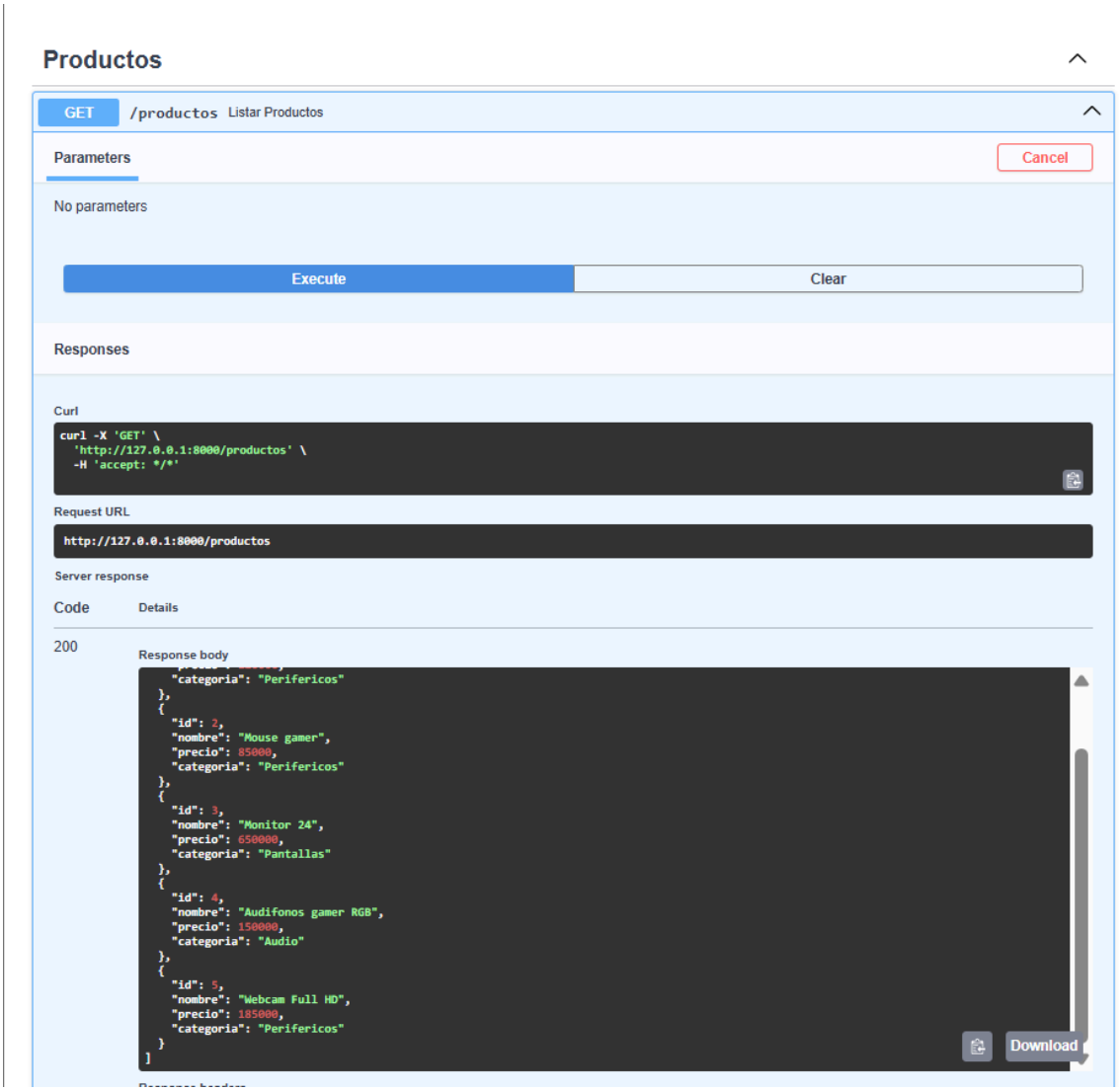
Solicitud POST para crear Webcam Full HD



Nota. Se registró un segundo producto con precio 185000 y categoría Perifericos. La respuesta visible es HTTP 201, correspondiente a la creación del recurso.

Figura 5

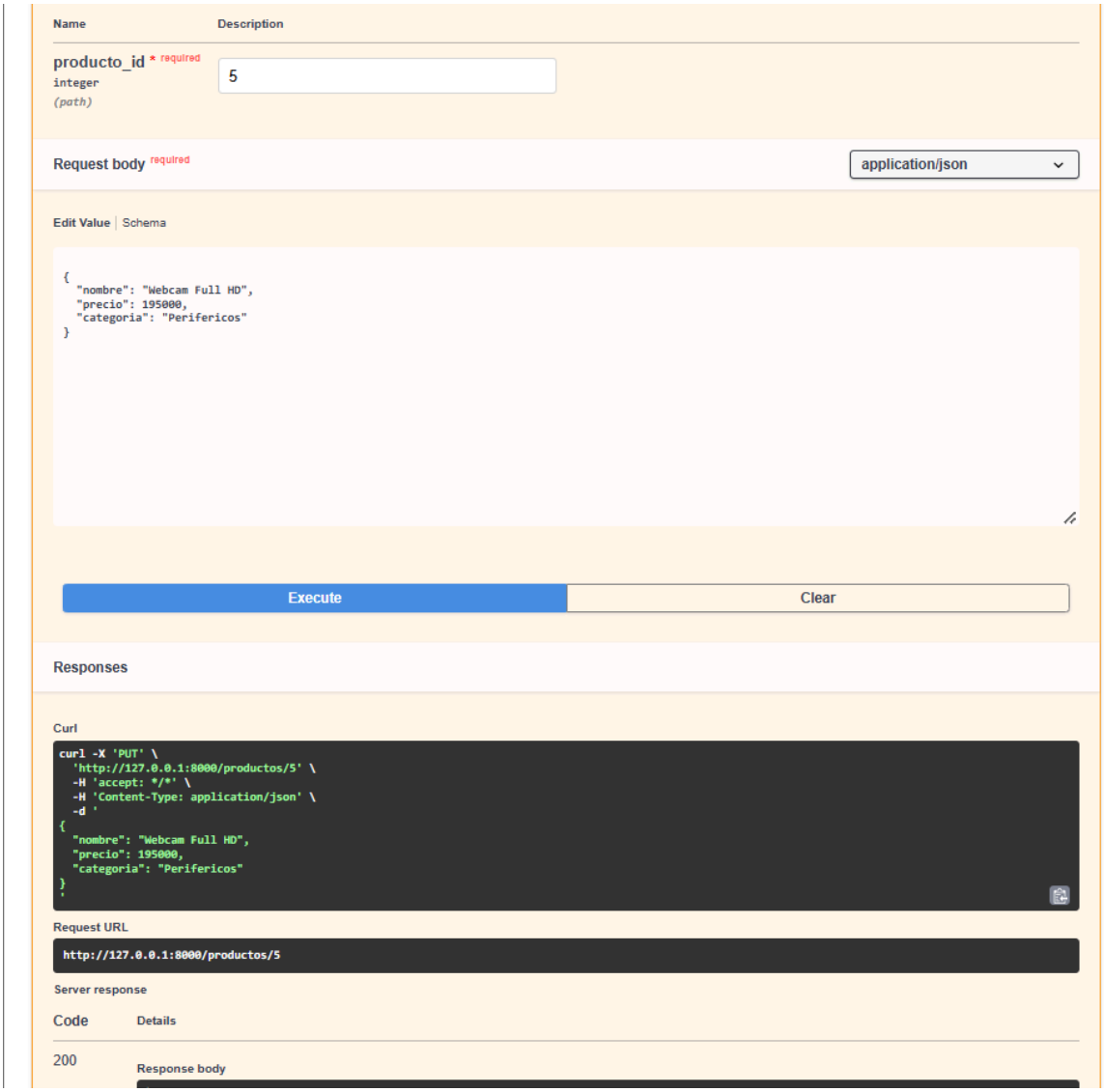
Consulta GET del listado con cinco productos



Nota. El servidor respondió HTTP 200 y mostró los tres registros iniciales y los dos creados durante la práctica, con identificadores 4 y 5.

Figura 6

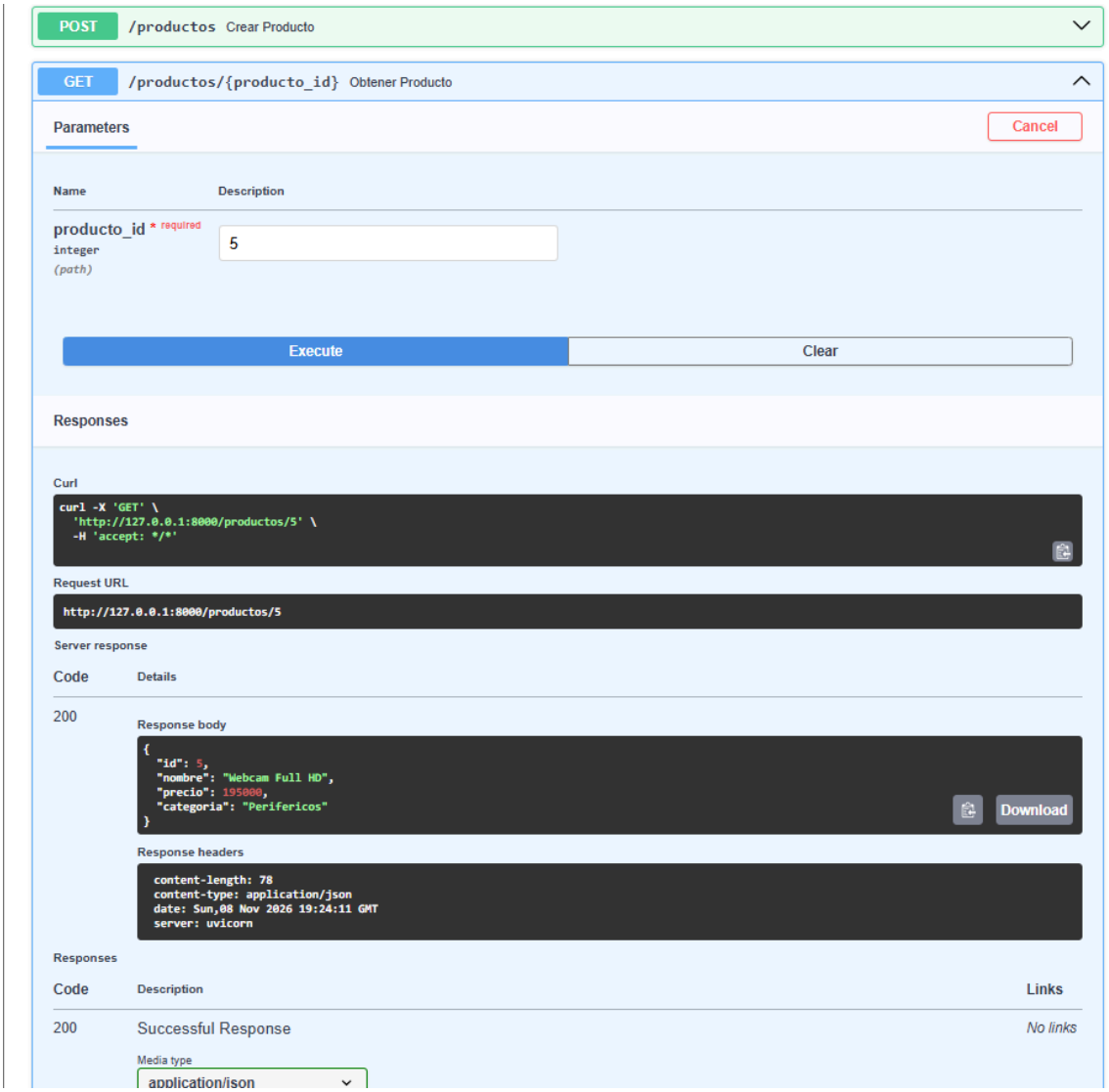
Actualización PUT del precio de Webcam Full HD



Nota. Se realizó PUT /productos/5 con precio 195000 y se obtuvo HTTP 200; la verificación posterior mediante GET se presenta en la siguiente figura.

Figura 7

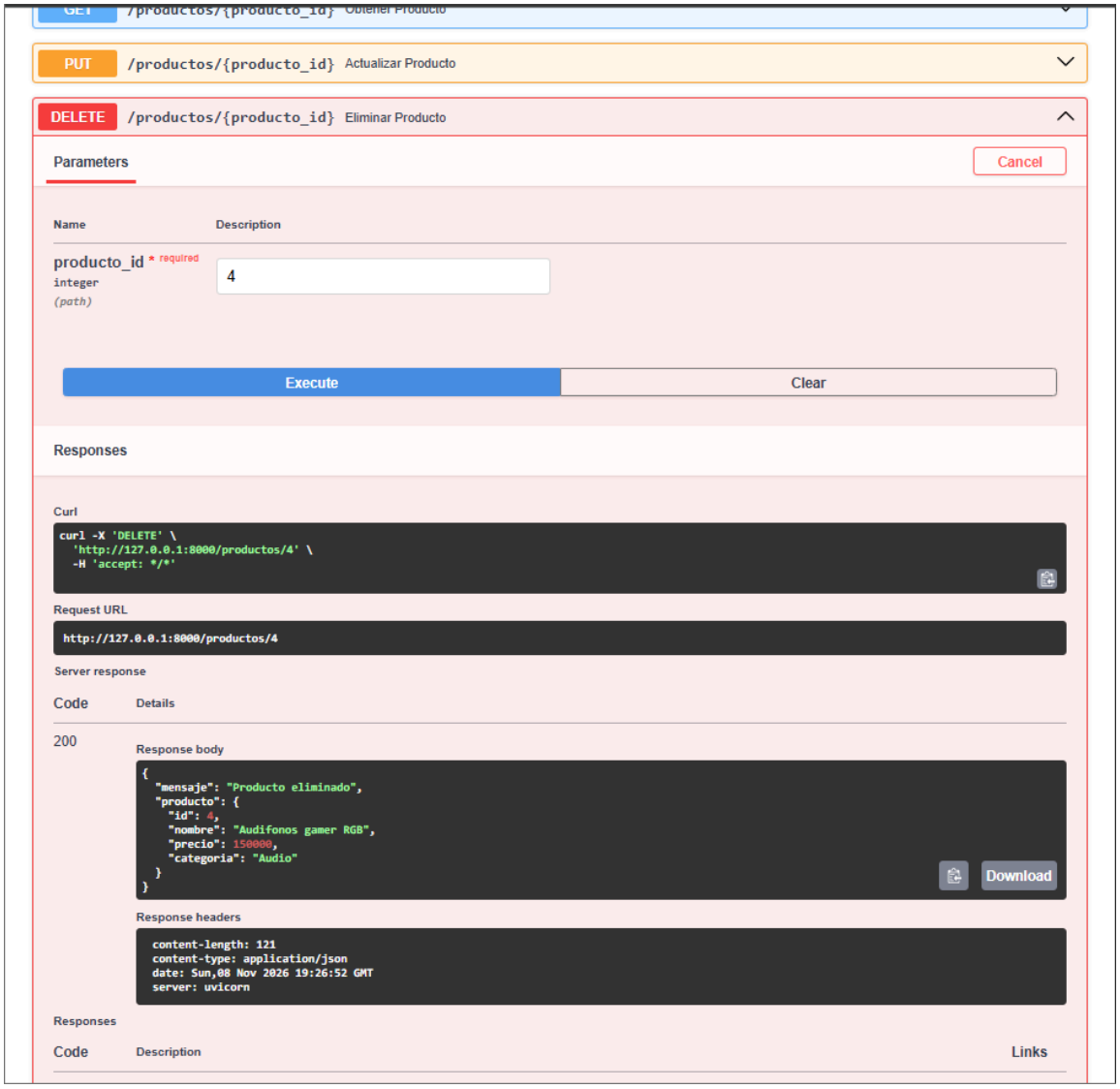
Verificación GET de producto actualizado



Nota. La consulta GET /productos/5 devolvió HTTP 200 con el precio actualizado a 195000, confirmando la modificación en memoria.

Figura 8

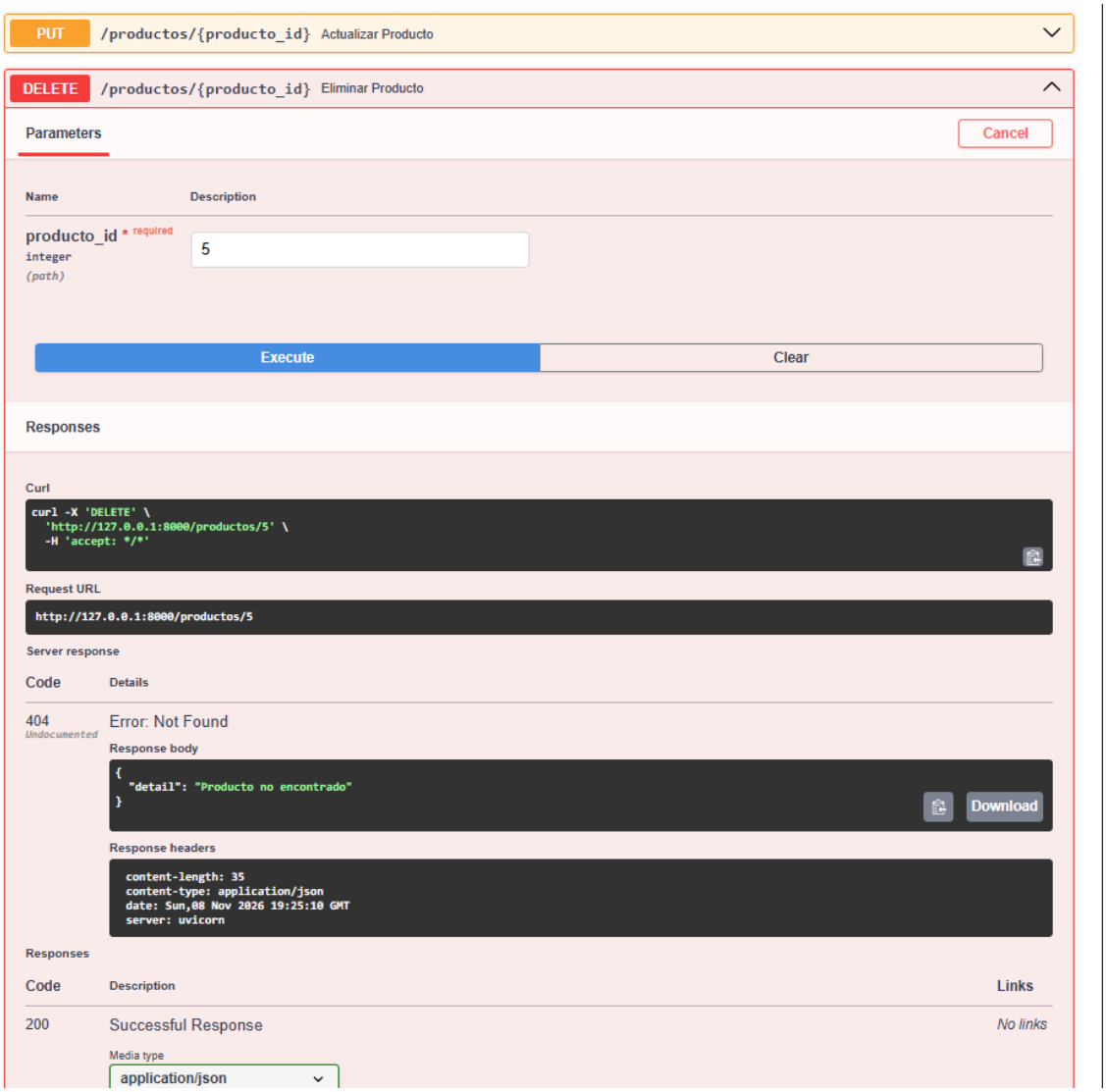
Eliminación exitosa mediante DELETE



Nota. La operación DELETE /productos/4 respondió HTTP 200 con el mensaje «Producto eliminado». La ausencia del ID 4 tras el borrado se confirma mediante una prueba independiente descrita en el análisis de resultados.

Figura 9

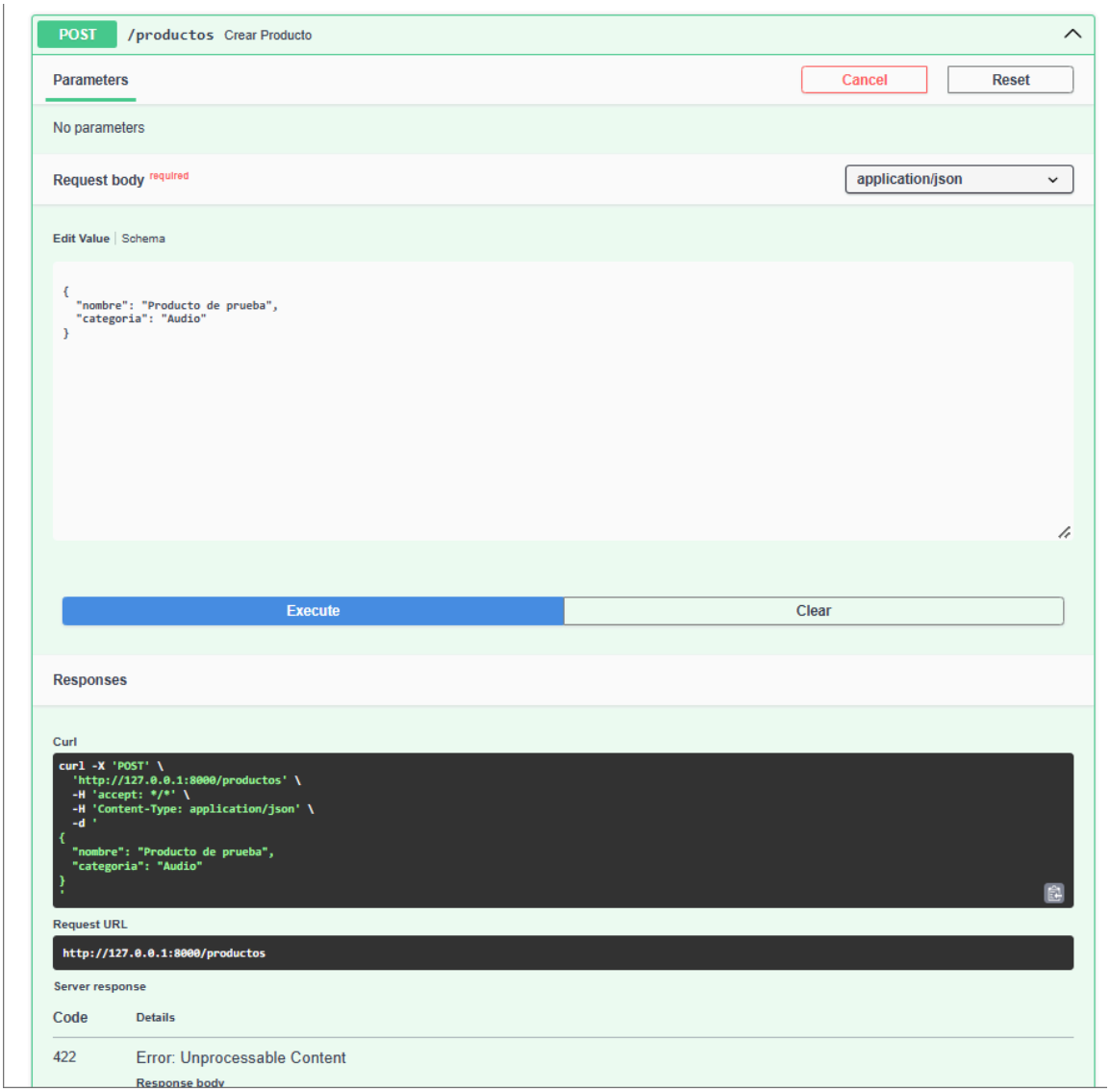
Manejo del error 404 en la eliminación de un producto inexistente



Nota. La captura muestra que DELETE /productos/5 respondió HTTP 404 con el detalle «Producto no encontrado». Esta solicitud se presenta como prueba del manejo de identificadores inexistentes y no como continuación de la secuencia de las Figuras 7 y 8.

Figura 10

Validación de datos incompletos en POST /productos



Nota. Al omitir el campo obligatorio precio se obtuvo HTTP 422. La captura permite verificar el código de respuesta; el detalle JSON de validación queda fuera del recorte.

GA2-AA3-EV04. CRUD de categorías

Como reto de transferencia se implementó el módulo categorias.py, tomando como referencia el funcionamiento de productos.py. El modelo CategoriaEntrada define únicamente nombre: str y el servidor asigna un identificador a cada nueva categoría. Se conservaron los registros iniciales Perifericos, Pantallas y Audio.

Tabla 3

Operaciones implementadas en el recurso categorías

Operación	Ruta	Respuesta esperada
GET	/categorias	Listado · 200
GET	/categorias/{categoria_id}	Categoría · 200 / 404
POST	/categorias	Creación · 201 / 422
PUT	/categorias/{categoria_id}	Actualización · 200 / 404 / 422
DELETE	/categorias/{categoria_id}	Eliminación · 200 / 404

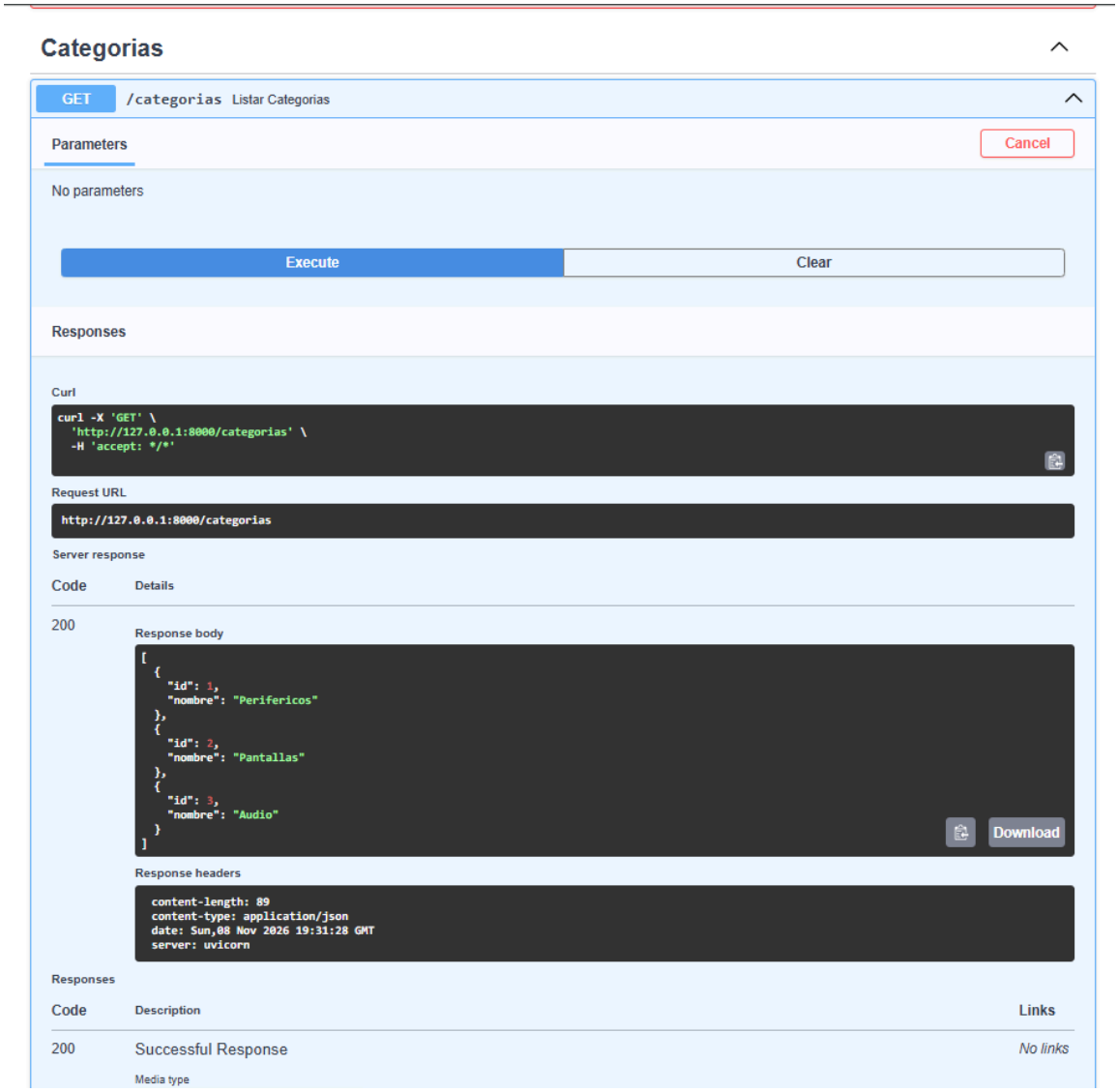
Nota. Elaboración propia con base en el router de categorías implementado durante la actividad.

Las pruebas se realizaron con datos de ejemplo. Primero se listaron las categorías, se añadió Accesorios Gamer (ID 4), se consultó y se cambió su nombre a Accesorios Gaming. Después se eliminó el registro y se comprobó la respuesta 404; finalmente se envió un JSON vacío para provocar la validación 422.

Pruebas y evidencias del recurso categorías

Figura 11

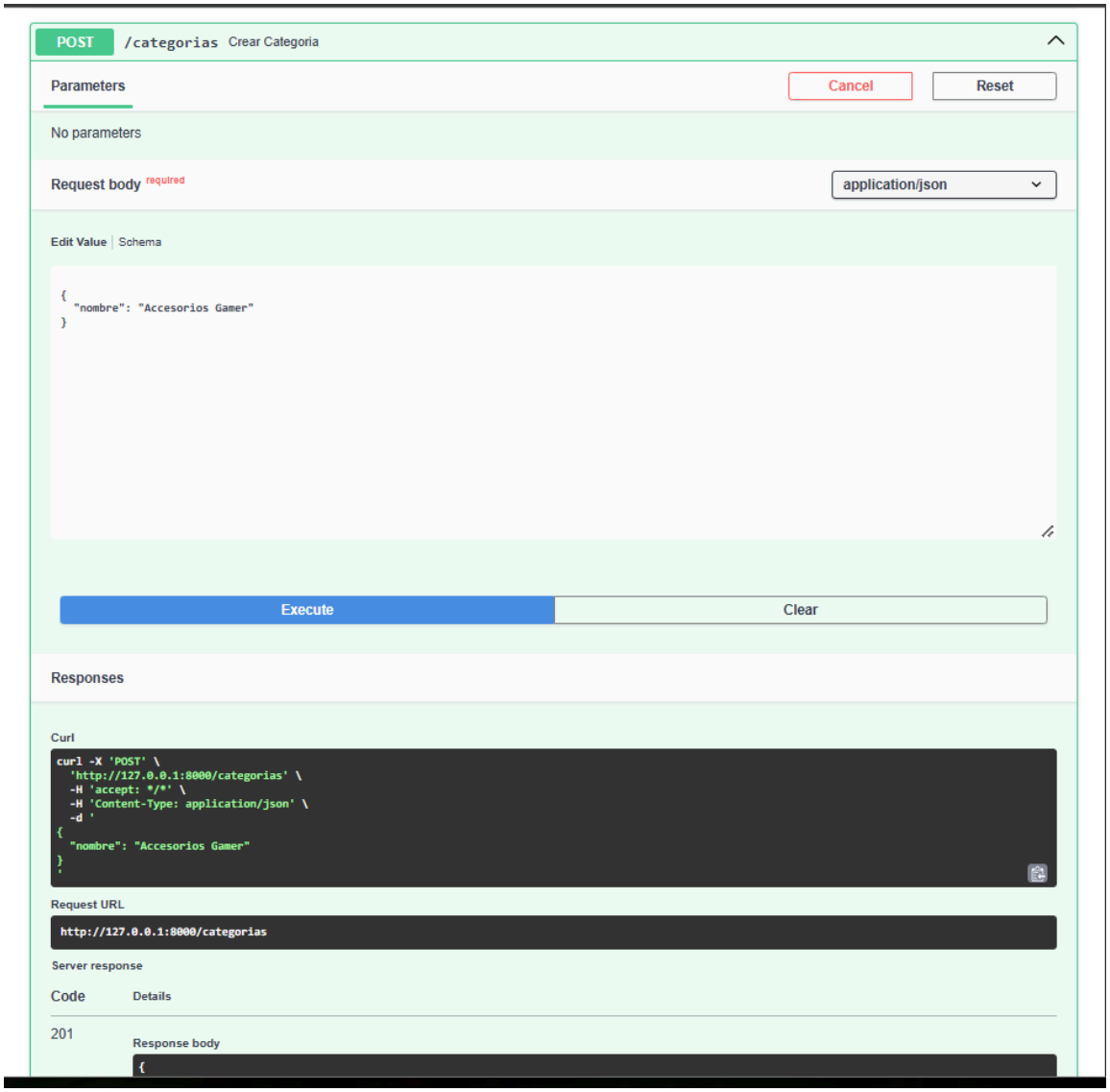
Listado inicial de categorías mediante GET



Nota. GET /categorias respondió HTTP 200 y devolvió las categorías de ejemplo: Perifericos, Pantallas y Audio.

Figura 12

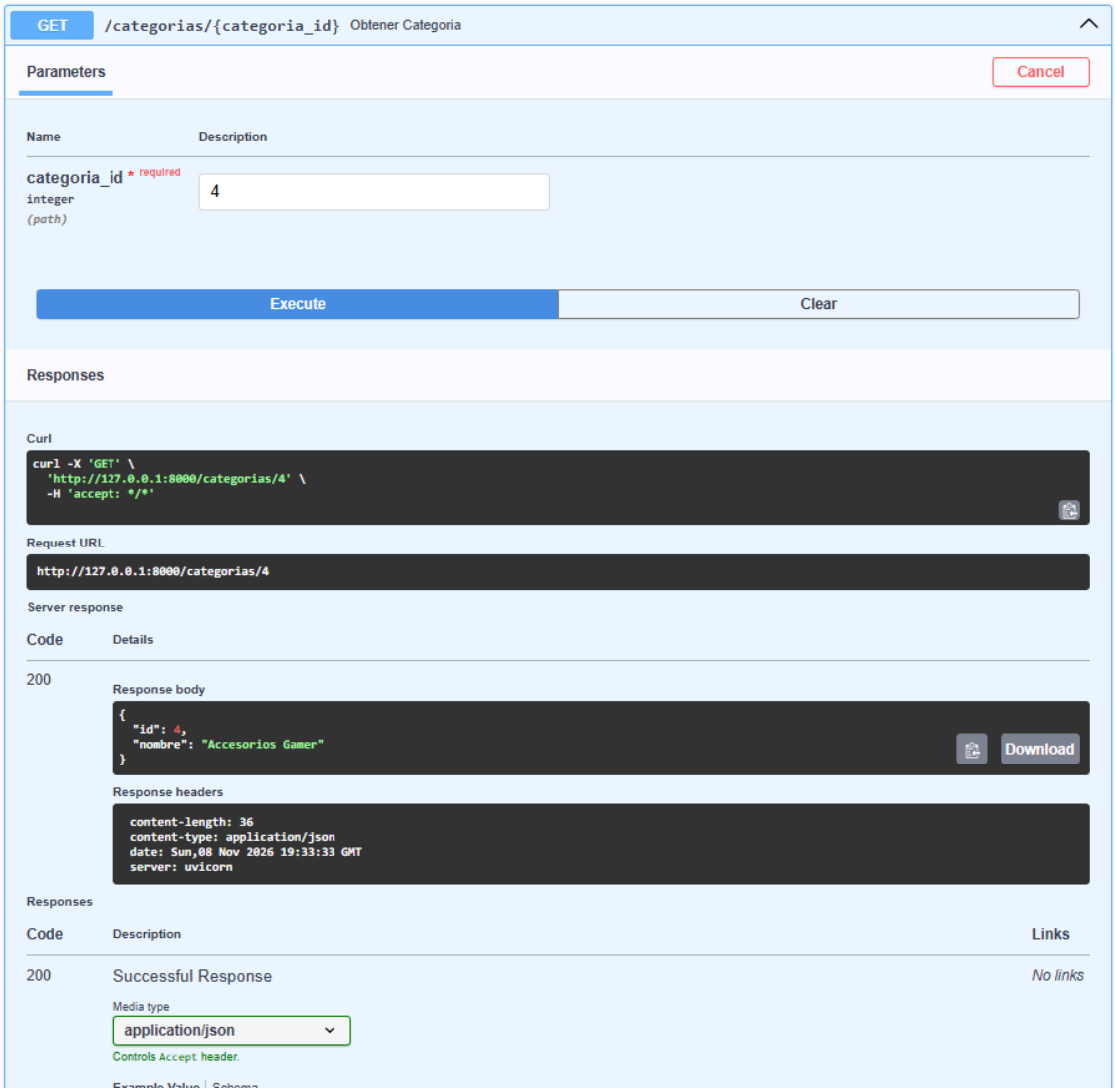
Creación de categoría mediante POST



Nota. Se envió el nombre Accesorios Gamer a POST /categorias y el servidor respondió HTTP 201. El cuerpo completo de respuesta no se aprecia en el recorte.

Figura 13

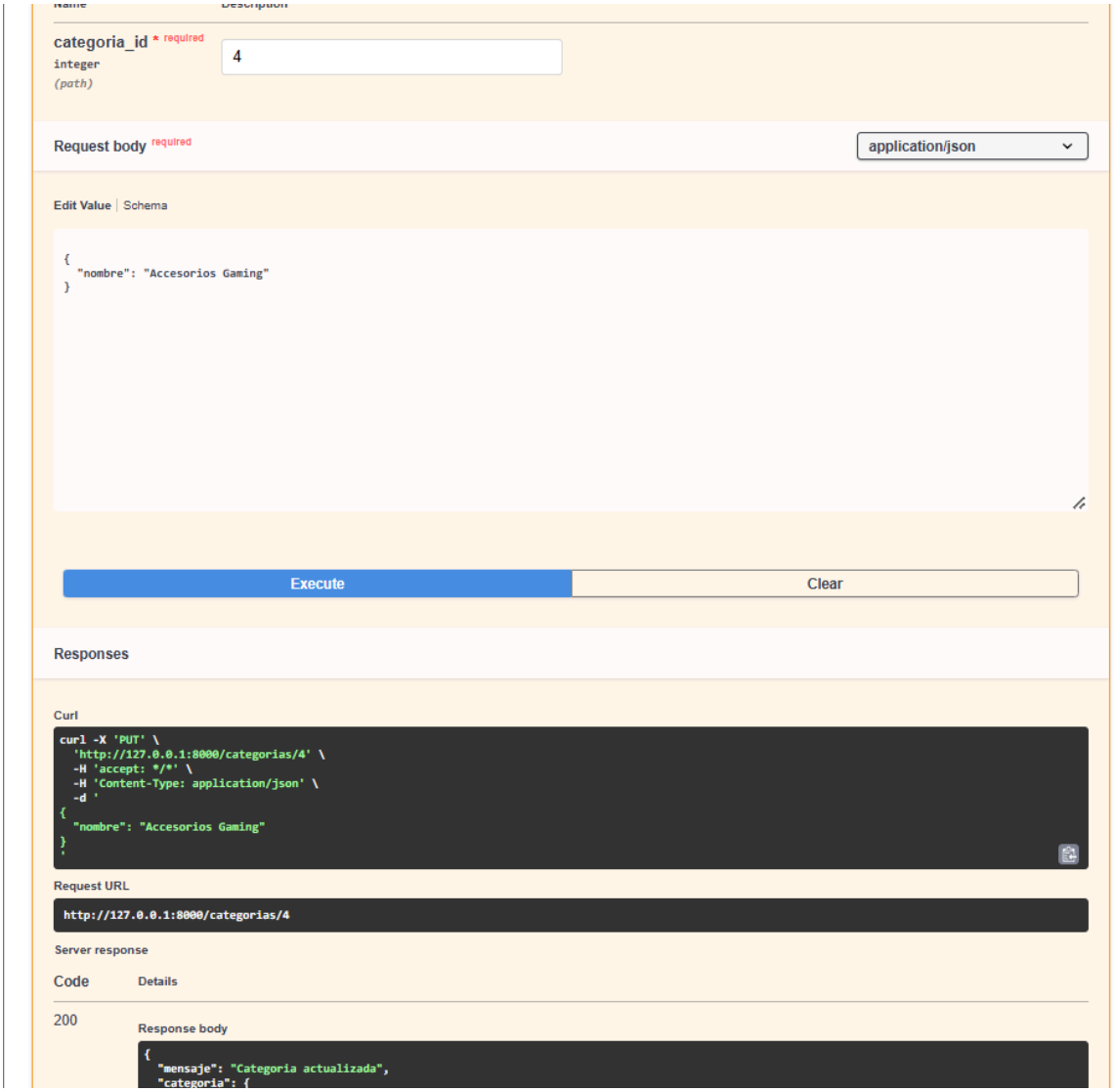
Consulta individual de la categoría creada



Nota. GET /categorias/4 devolvió HTTP 200 e identificó el registro con el nombre Accesorios Gamer.

Figura 14

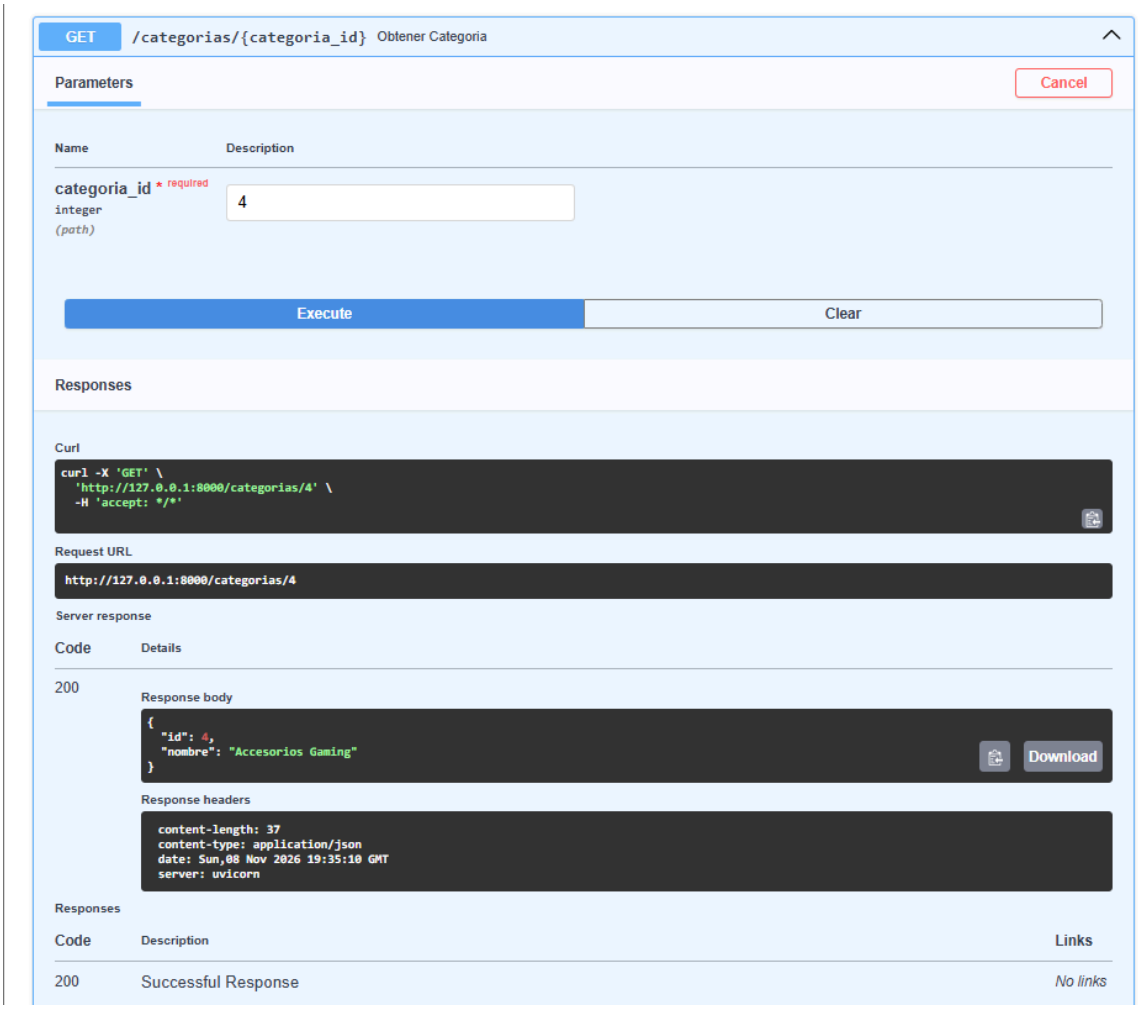
Actualización de una categoría mediante PUT



Nota. PUT /categorias/4 recibió el nuevo nombre Accesorios Gaming y respondió HTTP 200 con el mensaje Categoria actualizada.

Figura 15

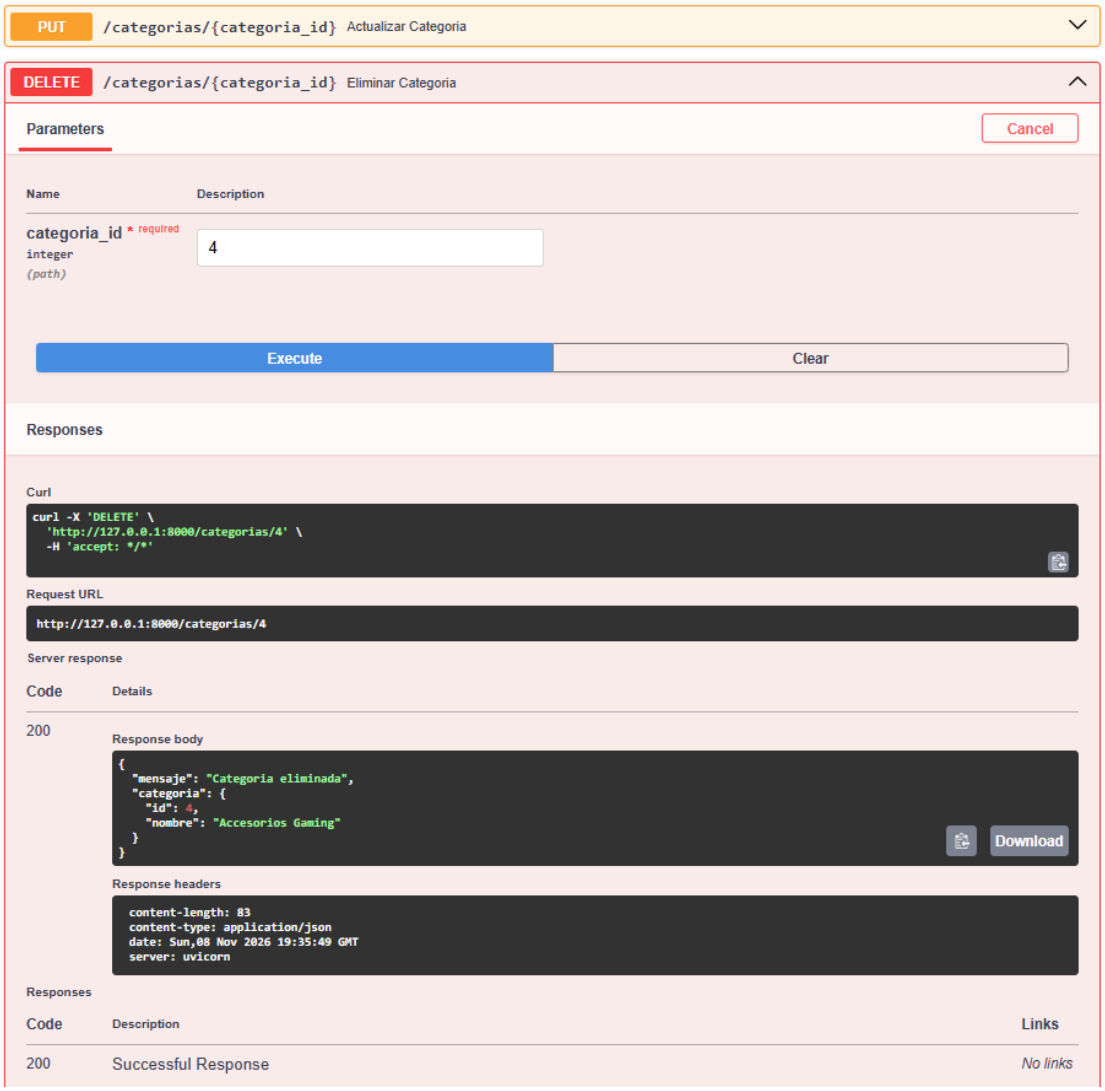
Confirmación de la actualización de categoría



Nota. GET /categorias/4 devolvió HTTP 200 y el nombre Accesorios Gaming, lo que confirma la modificación.

Figura 16

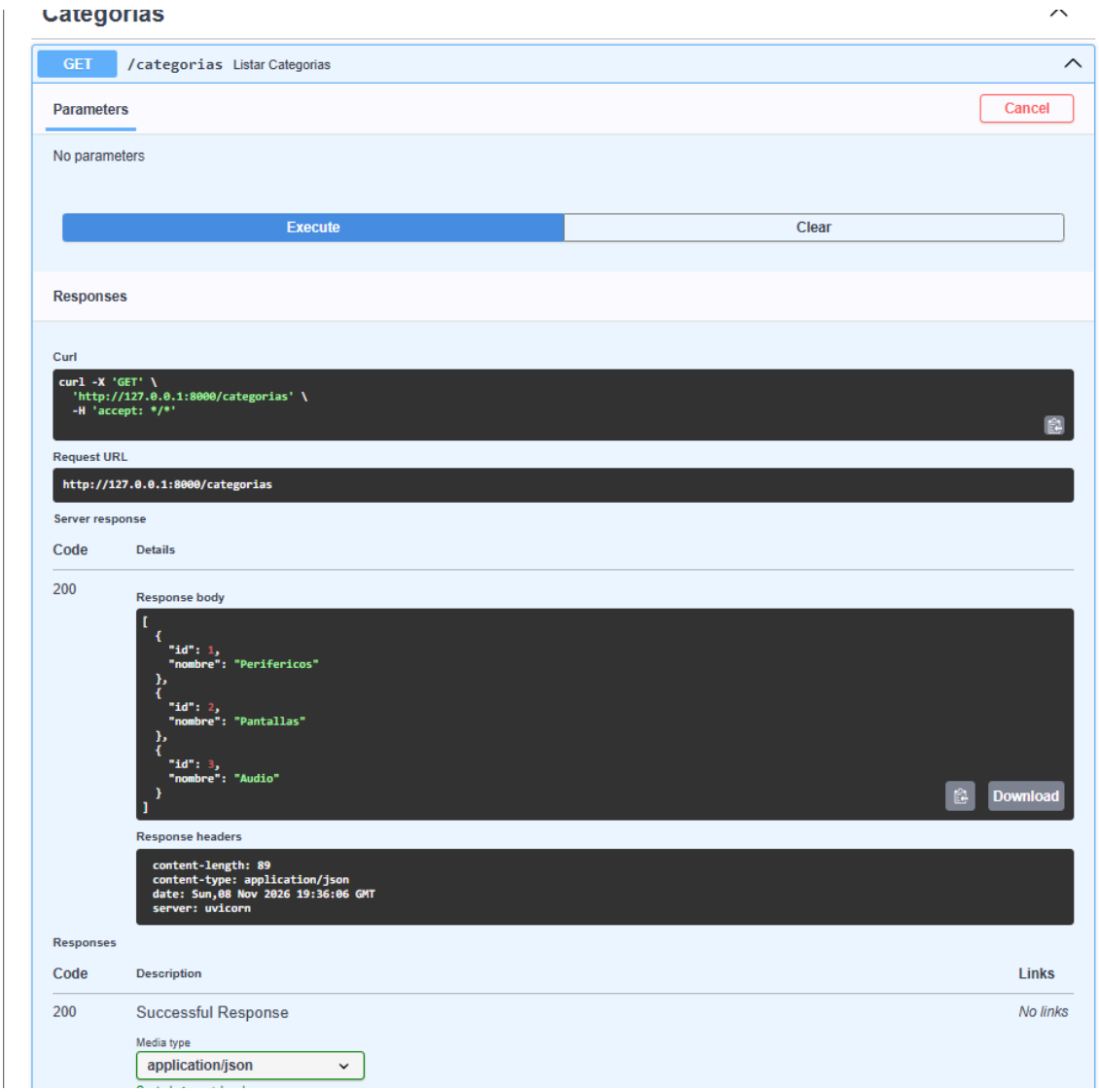
Eliminación exitosa de la categoría de prueba



Nota. DELETE /categorias/4 devolvió HTTP 200 y el mensaje Categoria eliminada, identificando el recurso retirado.

Figura 17

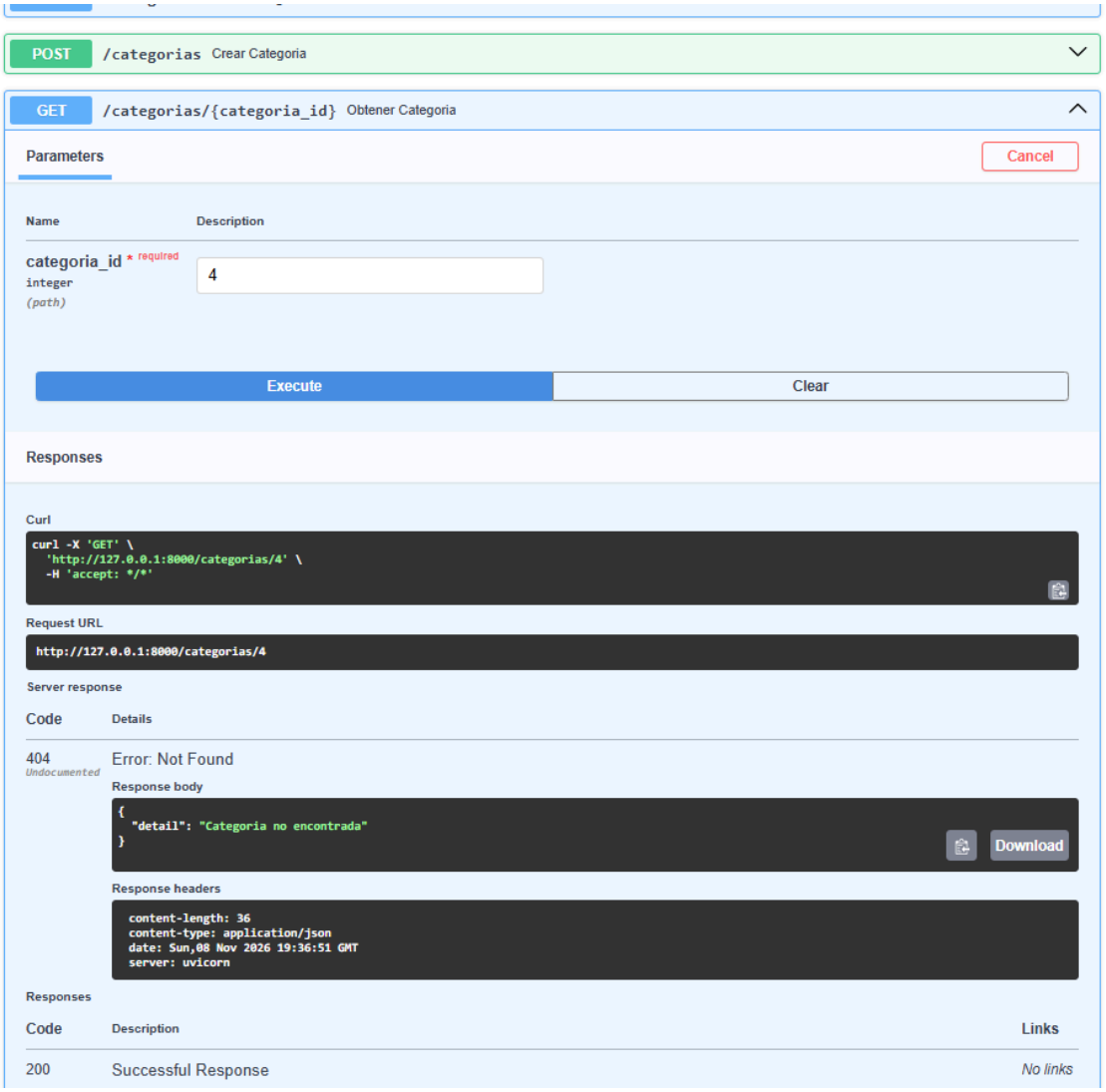
Listado de categorías después de DELETE



Nota. GET /categorias respondió HTTP 200 con solo las tres categorías iniciales; Accesorios Gaming ya no se encontraba en memoria.

Figura 18

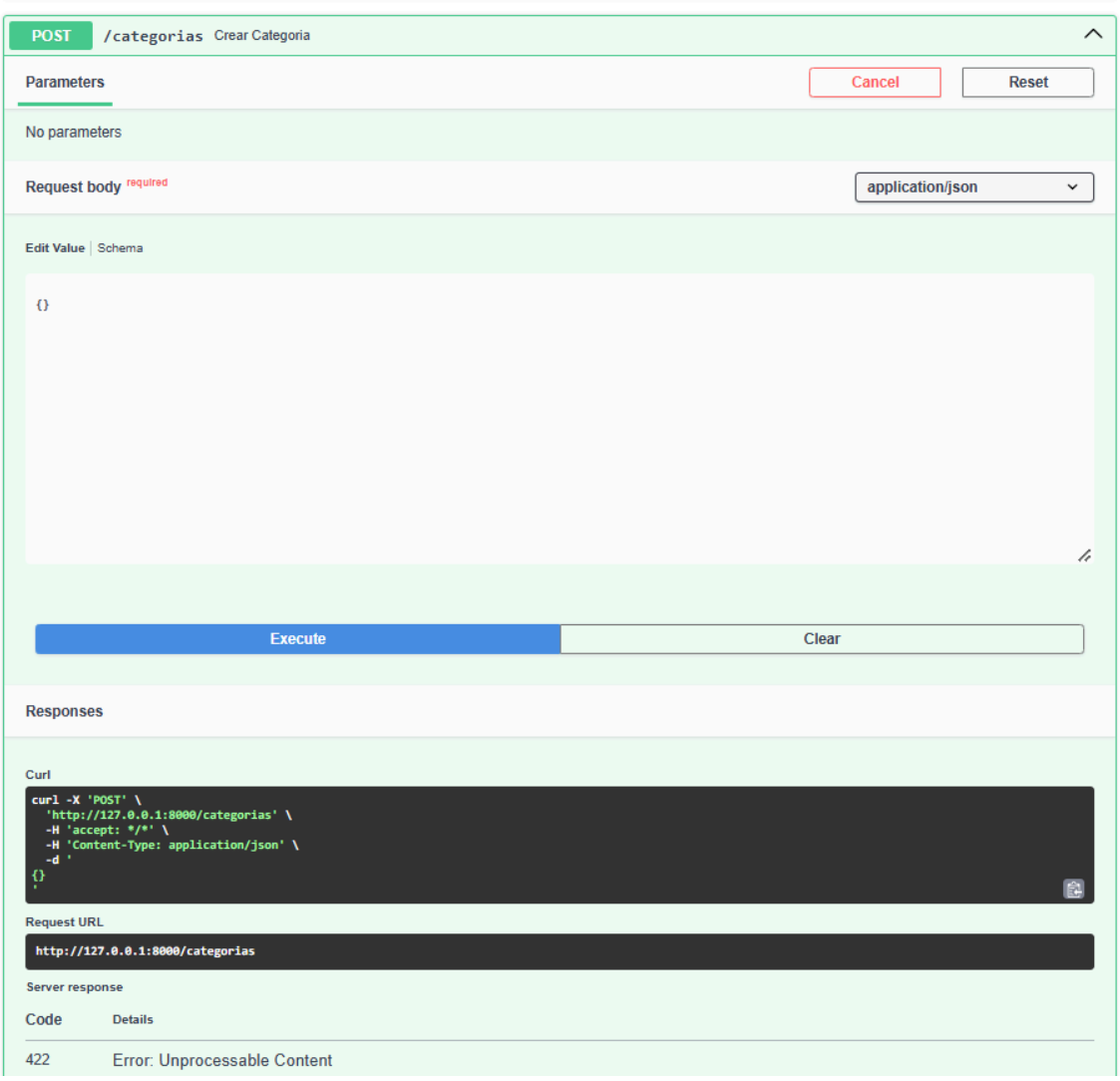
Error 404 al consultar una categoría eliminada



Nota. GET /categorias/4 devolvió HTTP 404 y el detalle Categoria no encontrada, evidenciando el manejo controlado del recurso inexistente.

Figura 19

Validación 422 para el modelo de categorías



Nota. POST /categorias recibió un objeto JSON vacío y devolvió HTTP 422. El cuerpo de respuesta no aparece en la captura, pero la omisión de nombre incumple el modelo CategoriaEntrada.

Análisis de resultados y comprobación técnica

Las capturas de los dos recursos permiten verificar respuestas exitosas de creación (201), lectura, actualización y eliminación (200), así como el manejo de datos inexistentes (404) y solicitudes incompletas (422). En los endpoints de escritura, FastAPI recibe JSON y aplica los modelos de Pydantic antes de procesar la información.

En el equipo de desarrollo se ejecutó python -m pytest tests -q desde el entorno virtual. La salida reportó 6 pruebas aprobadas, ninguna fallida y una advertencia de deprecación de dependencias del entorno de pruebas. La advertencia no impidió la ejecución correcta de la suite. Esta comprobación automatizada complementó las solicitudes manuales ejecutadas en Swagger UI.

Verificación adicional del borrado. En una prueba aislada con TestClient sobre una copia del código del informe, POST /productos creó el ID 4 (201); DELETE /productos/4 lo eliminó (200); GET /productos confirmó su ausencia (200) y GET /productos/4 devolvió 404. Esta prueba complementa Swagger UI y no corresponde a la misma sesión.

Los cambios son temporales: las colecciones de productos y categorías están almacenadas en listas dentro del proceso Python. Al reiniciar el servidor se restablecen los registros declarados en los módulos. Por tanto, la práctica cumple el propósito introductorio de la Guía 2, pero aún no incorpora persistencia ni integridad referencial en una base de datos.

La evidencia se conserva en el repositorio académico único de GitHub. La Guía 1 permanece separada y terminada en guia_01; todo el trabajo aquí documentado corresponde a guia_02. Repositorio: https://github.com/JuanEstebanLT/fastapi-sena.

Código fuente de la API modular

Los archivos principales se encuentran en guia_02/tienda_api/ del repositorio. A continuación se transcriben los módulos que definen la aplicación y sus operaciones. Se conserva la estructura simple establecida en la guía, con un modelo de entrada por recurso y datos almacenados en memoria.

Archivo guia_02/tienda_api/main.py

```
from fastapi import FastAPI
from guia_02.tienda_api.routers import productos, categorias

app = FastAPI(title="API de la Tienda", version="2.0.0")
app.include_router(productos.router)
app.include_router(categorias.router)

@app.get("/", tags=["Inicio"])
def inicio():
    return {"mensaje": "API de la Tienda funcionando. Visita /docs"}
```

Archivo guia_02/tienda_api/routers/productos.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

router = APIRouter(prefix="/productos", tags=["Productos"])

class ProductoEntrada(BaseModel):
    nombre: str
    precio: float
    categoria: str

productos = [
    {"id": 1, "nombre": "Teclado mecanico", "precio": 120000, "categoria": "Perifericos"},
    {"id": 2, "nombre": "Mouse gamer", "precio": 85000, "categoria": "Perifericos"},
    {"id": 3, "nombre": "Monitor 24", "precio": 650000, "categoria": "Pantallas"},
]

@router.get("")
def listar_productos():
    return productos

@router.get("/{producto_id}")
def obtener_producto(producto_id: int):
    for producto in productos:
        if producto["id"] == producto_id:
            return producto
    raise HTTPException(status_code=404, detail="Producto no encontrado")

@router.post("", status_code=201)
def crear_producto(datos: ProductoEntrada):
    nuevo_id = max((p["id"] for p in productos), default=0) + 1
    nuevo = {"id": nuevo_id, "nombre": datos.nombre,
             "precio": datos.precio, "categoria": datos.categoria}
    productos.append(nuevo)
    return {"mensaje": "Producto creado", "producto": nuevo}

@router.put("/{producto_id}")
def actualizar_producto(producto_id: int, datos: ProductoEntrada):
    for producto in productos:
```

```
        if producto["id"] == producto_id:
            producto["nombre"] = datos.nombre
            producto["precio"] = datos.precio
            producto["categoria"] = datos.categoria
            return {"mensaje": "Producto actualizado", "producto": producto}
    raise HTTPException(status_code=404, detail="Producto no encontrado")

@router.delete("/{producto_id}")
def eliminar_producto(producto_id: int):
    for producto in productos:
        if producto["id"] == producto_id:
            productos.remove(producto)
            return {"mensaje": "Producto eliminado", "producto": producto}
    raise HTTPException(status_code=404, detail="Producto no encontrado")
```

Archivo guia_02/tienda_api/routers/categorias.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

router = APIRouter(prefix="/categorias", tags=["Categorias"])

class CategoriaEntrada(BaseModel):
    nombre: str

categorias = [
    {"id": 1, "nombre": "Perifericos"},
    {"id": 2, "nombre": "Pantallas"},
    {"id": 3, "nombre": "Audio"},
]

@router.get("")
def listar_categorias():
    return categorias

@router.get("/{categoria_id}")
def obtener_categoria(categoria_id: int):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            return categoria
    raise HTTPException(status_code=404, detail="Categoria no encontrada")

@router.post("", status_code=201)
def crear_categoria(datos: CategoriaEntrada):
    nuevo_id = max((c["id"] for c in categorias), default=0) + 1
    nueva = {"id": nuevo_id, "nombre": datos.nombre}
    categorias.append(nueva)
    return {"mensaje": "Categoria creada", "categoria": nueva}

@router.put("/{categoria_id}")
def actualizar_categoria(categoria_id: int, datos: CategoriaEntrada):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            categoria["nombre"] = datos.nombre
            return {"mensaje": "Categoria actualizada", "categoria": categoria}
    raise HTTPException(status_code=404, detail="Categoria no encontrada")

@router.delete("/{categoria_id}")
def eliminar_categoria(categoria_id: int):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            categorias.remove(categoria)
            return {"mensaje": "Categoria eliminada", "categoria": categoria}
    raise HTTPException(status_code=404, detail="Categoria no encontrada")
```

Conclusiones

La Guía 2 permitió completar las operaciones CRUD de un catálogo de tienda. A diferencia de la Guía 1, que se enfocó en consultas GET, la nueva implementación incorpora POST, PUT y DELETE para administrar productos y categorías por medio de solicitudes HTTP.

La validación mediante Pydantic facilitó la detección de campos obligatorios ausentes. Las solicitudes incompletas originaron respuestas HTTP 422; por su parte, los identificadores inexistentes devolvieron HTTP 404 mediante HTTPException. Estas respuestas ayudan a identificar y comunicar errores de forma consistente.

El uso de APIRouter permitió separar los recursos en módulos especializados y mantener un archivo main.py breve, lo cual mejora la lectura del código y su mantenimiento. Las pruebas automatizadas y las evidencias de Swagger UI comprobaron que los endpoints principales respondían conforme a los escenarios de la actividad.

Se concluye que la solución satisface el alcance formativo del laboratorio modular y constituye una base apropiada para estudiar la persistencia de datos en guías posteriores. Los registros permanecen en memoria, por lo que no se afirma que la aplicación utilice una base de datos real.

Referencias bibliográficas

Arroyave, M. (2025). *Guía 2: Desarrollo de API REST con FastAPI* [Guía de aprendizaje]. Servicio Nacional de Aprendizaje (SENA).

Lezcano Tejada, J. E., Suárez, M., & Villa, E. (2026). *FastAPI-SENA: Guía 2, API de tienda con CRUD* [Código fuente]. GitHub. https://github.com/JuanEstebanLT/fastapi-sena

MDN Web Docs. (s. f.). *Métodos de petición HTTP.* https://developer.mozilla.org/es/docs/Web/HTTP/Methods

Pydantic. (s. f.). *Models.* https://docs.pydantic.dev/latest/concepts/models/

Ramírez, S. (2024a). *Bigger applications - Multiple files*. FastAPI. https://fastapi.tiangolo.com/tutorial/bigger-applications/

Ramírez, S. (2024b). *Request body*. FastAPI. https://fastapi.tiangolo.com/tutorial/body/

Control de versiones

Tabla 4

Registro de elaboración y revisión del documento

Versión	Fecha	Descripción	Responsables
1.0	08/11/2026	Documento inicial de la Guía 2, con las cuatro evidencias, pruebas y anexos técnicos.	Juan Esteban Lezcano Tejada, Mariana Suárez y Emanuel Villa

Nota. La versión corresponde al presente documento académico de la Guía 2. No modifica los archivos de la Guía 1.